

Utvecklingsmodell --- Visual Language v1

Status: Normativ v1 Renderer baseline: D2 0.9.0 Layout engine: ELK

Primary output: SVG Raster output: PNG derived from SVG

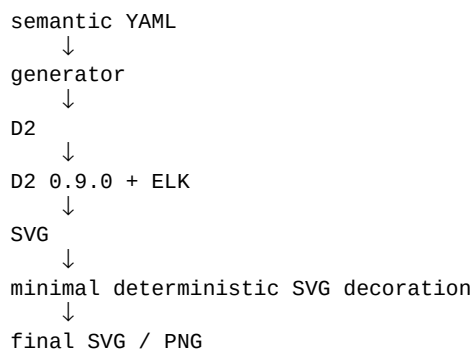
1. Purpose

Visual Language v1 defines the notation used to render the system artifacts Context, Pulse, and UI.

The semantic artifacts are authoritative. The visual language defines notation. D2 is only the renderer.

Rendering must be deterministic and reproducible. Semantics must never be weakened or changed to accommodate renderer limitations.

Canonical pipeline:



The same semantic input, Visual Language version, D2 version, and layout engine should produce the same diagram.

2. General principles

The symbol expresses what something is. The edge expresses the relationship. The text expresses the system-specific meaning.

Color is secondary. The notation must remain understandable in black and white.

Stable semantic IDs belong to the source artifacts. Display text may change without changing identity.

Generated D2 and SVG are derived artifacts; semantic YAML is the source.

3. Context

3.1 Purpose

Context describes: - the system being described; - people and external systems that communicate directly with it; - important domain data flows across the system boundary; - which party can initiate the interaction giving rise to a flow.

Context is not: - a detailed data-flow diagram; - a sequence diagram; - integration architecture; - an implementation or protocol diagram.

3.2 Elements

System: - exactly the system being described; - normally one; - rendered as a visually dominant rectangle with a stronger border.

Person: - a human role directly interacting with the system; - rendered with a person symbol.

External system: - an external technical system or service directly communicating with the system; - rendered as a rounded rectangle.

Internal implementation elements such as databases, containers, schedulers, internal HTTP servers, and internal services are excluded.

3.3 Data flows

A Context relation represents a domain data flow.

The relation name is a noun or noun phrase, for example: - Vehicle state - Electricity prices - Charging command - Energy reading

Protocol and implementation terms such as GET, POST, request, or response should not be used as the domain flow name.

If opposite directions carry different information, they are represented as separate flows.

3.4 Data direction

An arrowhead indicates only the direction in which the named data flows.

The arrowhead has no initiative semantics.

3.5 Initiative

An empty circle ■ indicates that the party at that end can initiate the interaction giving rise to the data flow.

No initiative circle means initiative is unspecified. It does not mean that nobody initiates.

Initiative and data direction are orthogonal semantic properties.

The initiative circle: - is centered exactly on the actual geometric connector path; - is placed near the initiating endpoint; - remains a separate symbol from the arrowhead; - must never visually merge with an arrowhead.

When initiative and an arrowhead occur at the same endpoint, there must be a small but clearly visible distance between ■ and the arrowhead, measured along the connector path.

Example:



not:



The two marks must not be perceived as a combined glyph.

3.6 Relation labels

A data-flow label is an annotation, not part of the connector.

The label must have clear visual separation from connector geometry.

Its visual bounding box must have a minimum clearance from all relevant connector segments. A connector must not pass through or immediately adjacent to the visible text.

Label placement must be based on actual rendered geometry rather than a fixed vertical offset.

3.7 Connector spacing

Parallel or nearby Context relations must have enough separation for: - their connector paths; - their labels; - initiative symbols; - arrowheads.

Layout spacing should be solved in the layout/generation stage where possible. Postprocessing must not compensate for fundamentally overcrowded relation routing by arbitrarily moving labels.

3.8 Rendering status

Context nodes: Native D2. Context connector routing: Native D2/ELK.

Initiative symbol: Adapted by minimal deterministic SVG decoration.

Label clearance: Adapted where necessary using rendered geometry.

The verified Context rendering establishes that initiative, data direction, and data-flow text are three visually distinct aspects.

4. Pulse

4.1 Purpose

Pulse describes that event chains start and how they can propagate through the system.

It describes possible causal event flows.

Pulse does not describe: - implementation; - execution logic; - data dependencies; - exact runtime traces.

Using, reading, or depending on data does not by itself create Pulse semantics.

4.2 Semantic elements

Pulse has three fundamental semantic concepts: - Behavior - Pulse - Flow

Trigger is a role played by an external or starting cause. It is not a separate permanent semantic element.

Behavior: - something the system does in reaction to a Pulse; - rendered as a rounded rectangle.

Pulse: - an identified event that can drive a Behavior; - has a stable semantic ID; - may also have a short display identity such as 01.

Flow: - introduces or emits a Pulse and directs it to a Behavior.

A Behavior may emit zero, one, or several Pulses. An outgoing Pulse means "can emit", not "always emits". Conditions remain internal to Behavior.

Multiple incoming Pulses have independent/OR semantics. The same Pulse may fan out to several Behaviors. Cycles are allowed. No special gateway, end, or loop notation is required.

4.3 Trigger role

A trigger is a free domain description of a cause that introduces a Pulse into the event chain, for example: - Startup - Vehicle observation due - Select target SoC

Identical trigger text should render as one visual trigger source with

fan-out where applicable.

4.4 Pulse notation

A Pulse is shown as a numbered circle integrated into the connector:

source (01)  target

The Pulse circle is not a connector label.

Normative geometry rule:

The center of the Pulse symbol must lie exactly on the geometric path of the Pulse connector.

The symbol is an integrated part of the connector and must not float above or beside it.

The same semantic Pulse uses the same display number wherever it appears.

Example:

A (02)  B (02)  C

A different Pulse receives a different display identity:

A (02)  B (03)  C

Fan-out repeats the Pulse symbol on every branch.

4.5 Pulse legend

A diagram may include a Pulse legend:

PULSES

01 Vehicle observation
02 Vehicle state updated
03 Price retrieval

The short number is display identity only. Cross-references use the semantic Pulse ID.

4.6 Rendering status

Behavior: Native D2. Connector routing and arrowhead: Native D2/ELK.

Pulse symbol: Adapted by minimal deterministic SVG decoration.

The decoration must derive the Pulse symbol position from the actual SVG connector path.

5. UI

5.1 Purpose

UI describes user-visible system capability in terms of: - Views; - Information; - Actions; - meaningful Navigation between Views.

UI does not describe: - layout details; - responsive design; - colors; - typography; - widgets or form controls; - gestures; - scrolling; - animation; - technical components; - API calls.

A useful granularity test is:

If two UI elements can be redesigned independently without changing the system's user function, they are probably presentation and should not be separate View elements.

5.2 View

A View is a user-relevant system surface.

It is rendered as a rounded container with its name as the container title.

5.3 Information

Information represents a user-relevant concept shown by a View.

Its name is a noun or noun phrase and is independent of its UI representation.

User-relevant status, errors, and indicators may be Information when they represent meaningful concepts. Individual visual states are not separately modeled merely because the UI renders them differently.

Information is rendered as:

■ Information name

5.4 Action

An Action represents an intentional user domain action that affects the system or the user's work state.

Its name is a verb phrase.

Actions model user intent, not gestures or controls.

For example: - Select target SoC - Select deadline - Return to automatic control

Scrolling, swiping, opening a tooltip, or tapping a particular widget are not Actions unless the domain intent itself is significant.

Action is rendered as:

■ Action name

5.5 Information and Action layout

Within a View: - Actions are placed in the left column. - Information is placed in the right column. - Items within each column follow the order declared in the semantic artifact. - No separate ACTIONS or INFORMATION headings are used. - ■ and ■ carry the distinction.

This is a normative Visual Language rule, not a statement about the application's actual screen layout.

A View with no Actions naturally renders as an Information-only View.

5.6 Navigation

Navigation is the only explicit relation between Views.

Navigation is modeled only when the path between Views has user significance.

Generic/global access among main application Views need not be represented as a complete navigation graph.

Navigation describes the meaningful relationship, not menu, tab, button, or other navigation implementation.

When rendered, Navigation uses a lighter/dashed directed relation so it remains visually secondary to View contents.

5.7 Rendering status

View container: Native D2. Information and Action symbols/text: Native D2. Two-column layout: Native/generated D2 structure with deterministic ordering. Navigation: expected Native D2 when present.

No SVG postprocessing is currently required for the verified UI notation.

6. Artifact-specific semantic sources

Visual Language v1 intentionally does not define a generic metamodel.

The preferred source artifacts are three small artifact-specific YAML formats:

```
system/  
context.yaml
```

```
pulse.yaml
ui.yaml
```

Generated artifacts:

```
context.yaml → context.d2 → context.svg
pulse.yaml   → pulse.d2   → pulse.svg
ui.yaml      → ui.d2      → ui.svg
```

Artifact-specific vocabulary is intentionally asymmetric and should remain understandable to a human reader.

7. Identity

Machine-readable IDs should be simple and stable within their natural artifact scope.

Do not introduce UUIDs, namespaces, or a global identity registry in v1.

name is display text and may change while id remains stable.

Pulse additionally has a short display identity such as 01; that display identity is not its semantic identity.

Cross-artifact references are not normative in v1.

Semantic duplication between artifacts is acceptable in v1 rather than prematurely introducing cross-artifact reference machinery.

8. Renderer acceptance classes

Renderer support is classified as:

Native: The notation can be rendered directly and deterministically by D2.

Adapted: A small deterministic generated construction or SVG decoration is required.

Unsupported: The notation would require manual positioning or substantial unstable SVG manipulation.

Current v1 classification:

Feature Classification

System / External system / Person Native

UI Native

Information Native

Action Native

Behavior Native

D2/ELK connector routing Native

Pulse symbol on connector Adapted

Context initiative circle Adapted

Context label clearance Adapted where required

No current Visual Language v1 requirement justifies a custom full renderer.

9. Determinism and implementation constraints

The renderer implementation must: - pin the D2 version; - pin the layout engine; - generate D2 deterministically from semantic YAML; - perform only deterministic SVG decoration; - derive semantic-symbol placement from actual rendered connector geometry where required; - avoid manual per-diagram pixel fixes; - avoid image-AI redrawing; - preserve semantic meaning independently of renderer limitations.

Minimal SVG decoration is acceptable when it implements a stable Visual Language rule.

It must not become an uncontrolled second layout engine.

10. Verified v1 decisions

The following have been visually tested and accepted:

Context: - system/external-system composition; - domain data-flow labels; - independent arrowhead for data direction; - initiative circle centered on connector path; - visible separation between initiative circle and arrowhead; - connector/label clearance; - sufficient relation spacing.

Pulse: - left-to-right causal flow using ELK; - Behavior nodes; - numbered Pulse circle integrated into connector; - Pulse circle centered on actual connector path.

UI: - View containers; - ■ Actions; - ■ Information; - Actions in the left column; - Information in the right column; - deterministic declared ordering within each column.

These verified rendering decisions constitute Visual Language v1.

Requirement indicators - normative v1 addendum

Date: 2026-09-18. Status: Normative supplement to Visual Language v1.

This addendum defines requirement indicators and interactive access to requirements. Context, Pulse and UI notation remains unchanged. The renderer baseline remains D2 0.9.0 with ELK. Requirement targets and source format follow the companion Artifact Formats Requirements addendum.

R1. Purpose and meaning

A requirement indicator shows that requirements are directly attached to a model element. It is a review annotation and access control for those requirements, not a new semantic model element, data flow, Pulse or Action.

The indicator is a small outlined circle containing the lowercase letter `r`. The circle has a clear, opaque background. The letter is centered and readable. Meaning must remain clear in black and white; color is optional and secondary.

Show an indicator if and only if the element has a resolved, non-empty requirement attachment. Do not show indicators for elements without directly attached requirements. A container's indicator represents that container's own requirements, not the union of its contents' requirements.

R2. Binding and repeated occurrences

Each indicator binds to exactly one target address from `requirements.yaml`. Activating it opens the requirements for that target. Never bind by label text or Pulse display number.

One indicator is shown per rendered occurrence of the addressed element. Where the same Pulse is drawn on several branches, each occurrence receives the indicator if that Pulse has attached requirements. All such indicators open the same attachment.

R3. Placement

Placement must be deterministic and derived from actual rendered geometry. The indicator must appear close enough to its element that the attachment is unambiguous.

- Context flow: place it beside the data-flow label, clearly separated from both label and connector.
- System, party or Behavior: place it near the label or in a clear corner of the shape. Avoid text and connector attachment points.
- Pulse: place it beside the numbered Pulse symbol, outside that symbol and away from the connector path. Never put `r` inside the numbered circle.
- View: place it beside the View title. This represents requirements attached directly to the View.
- Action or Information: place it beside the item's label. Keep the existing Action and Information symbols visible.

An indicator must not overlap another indicator, any label, arrowhead, initiative symbol, numbered Pulse symbol or unrelated element. It must not obscure a connector or appear as part of that connector. Check clearance against the actual geometry of nearby elements. Use a consistent size and minimum clearance across diagrams; do not introduce manual per-diagram pixel fixes.

If sufficient space is unavailable, adjust deterministic generation or layout spacing. Do not weaken model notation or introduce an unstable second layout engine. Keep indicators inside the exported viewport without clipping.

R4. Interactive behavior

In the interactive review view, clicking an $\times$ circle must display the complete requirements directly attached to its element. A side panel or equivalent review surface is acceptable. Display the element's name, stable target address and every requirement string in declared order, without paraphrasing or omission.

The active target must be clear. Selecting another indicator replaces the displayed requirement list with that target's requirements. An Action or Information indicator must open its own requirements rather than those of its containing View.

Indicators must also be keyboard-focusable, have a visible focus state and activate with Enter or Space. Provide an accessible name identifying the element and the purpose of opening its requirements. Hover may provide supplementary help but must not be the only access mechanism. Pointer hit areas may be larger than the visible circle if they do not interfere with nearby controls.

Clicking the model element itself may additionally open its requirements. This is optional and does not replace the visible $\times$ indicator.

R5. Export and rendering

Generate indicators as minimal, deterministic SVG decoration after D2/ELK rendering. Do not add badge fields to semantic YAML or model the indicators as layout nodes. Target bindings derive from validated requirement attachments; positions derive from rendered element geometry.

Interactive behavior is guaranteed by the interactive review surface, which may embed SVG in HTML. A standalone SVG or static PNG need not offer click behavior. Static exports must retain visible indicators and provide a legend explaining that $\times$ denotes directly attached requirements. Static diagrams alone do not replace access to the requirements during review.

R6. Acceptance

- Every rendered occurrence with directly attached requirements has one readable $\times$ indicator; occurrences without attachments have none.
- Indicators preserve all existing model semantics and notation, including Pulse-circle and initiative geometry.
- No indicators, labels or semantic symbols collide or become clipped at the normal review size.
- Every indicator opens exactly its target's complete, ordered requirements. Repeated Pulse occurrences open the same target; child View items retain their own targets.
- Pointer and keyboard activation work, and keyboard focus is visible.
- Given identical semantic sources, requirements, renderer version and layout settings, placement and bindings are reproducible.